

**Document number:** P2570R2  
**Date:** 2023-01-14  
**Audience:** SG21  
**Reply-to:** Andrzej Krzemiński <akrzemi1 at gmail dot com>

# Contract predicates that are not predicates

In logic, a *predicate* is a symbol that represents a property or a relation. A contract annotation, as proposed in [P2521R2], requires an expression of type `bool`. In this paper we explore different possible semantics of programs where the expressions inside contract annotations do not satisfy the properties of a *predicate* from logic.

## 0. Revision history

{rev}

### 0.1. R0 → R1

{rev.r01}

1. Expanded the scope of the paper to include, apart from side effects, other properties of predicates, such as dependence on the externally modified state. This is reflected in the title change.
2. Shifted the discussion a bit from runtime evaluation of the predicates to static analysis based on the predicates.
3. Addressed the solution hinted at in [P2680R0]. See sections {dis.err} and {dis.mix}.
4. Described a solution where "pure" and "non-pure" contract conditions can coexist. See section {dis.mix}.
5. Discussed [N2956] (attribute `[[unsequenced]]`) as potentially useful in constraining contract predicates. See section {dis.uns}.
6. Described a criterion for determining to what extent the Standard Library components can be used in contract annotation. See section {ovr.cri.usa}.

### 0.2. R1 → R2

{rev.r02}

1. Expanded the Design Criteria section {ovr.cri}; in particular pointed out that we do not treat "UB-free" as synonymous with "safe".
2. Expanded on the example that demonstrates how the solution proposed in [P2680R0] has the potential to break correct code. See section {dis.err}.
3. Added section explaining why having side effects in contract predicates is an issue. See section {dis.why}.
4. Mentioned a future "preconditions-only" translation mode which may be impeded by the guarantee to evaluate side effects. See section {dis.sem}.
5. Added examples of other parts of the language where side effects are eliminated. See section {dis.sem.per}.

## 1. Overview

{ovr}

Because the choice of syntax for contract annotations has no consensus yet, in this paper we use a placeholder notation:

```
int select(int i, int j)
    PRE(is_even(i))           // precondition
    PRE(is_even(j))
    POST(r: is_even(r))       // postcondition; r names the return value
{
    ASSERT(is_nonnegative(_state)); // assertion statement

    if (_state == 0) return i;
    else return j;
}
```

Ideally, a contract annotation reads something like the above, and we intuitively assume that functions `is_even()` and `is_nonnegative()` are predicates in the logical sense, describing the properties of the input variables. Which implies that if we made the program to evaluate these predicates and discard the returned value, there would be no way to tell if these predicates were actually evaluated; and if we evaluated any such predicate two times, the two evaluations would give the same result. However, this simple intuition is gone if programmers put arbitrary expressions in contract predicates:

```
void f()
    PRE(++global_call_counter < 100);
```

Note that side effects may be hidden behind the function call:

```
void f()
    PRE(less_than_100_calls_so_far());

void turn_on_lights()
    PRE(it_is_evening()); // reads std::chrono::system_clock::now()
```

Note that side effects are only one problem, consider this example:

```
void g()
    PRE(global_thread_count < 32); // others can change it asynchronously
```

Because the above predicates are a possibility, we have to specify what the semantics of the resulting program are.

The range of possible choices includes:

- Do not use C++ expressions at all; instead invent a new mini-language where expressions are referentially transparent by design.
- Define the subset of C++ expressions as *referentially transparent*, with different meanings of "referentially transparent":
  - Make the program ill-formed if a contract predicate cannot be proven to be referentially transparent.
  - Have users declare their functions as referentially transparent, and compiler trust the users' declarations.
- Allow *any* expressions in contract predicates and let the tools decide how to deal with them:
  - For the purpose of static analysis only consume these expressions that can be proven to be referentially transparent, and ignore the others.
  - For runtime evaluation, just evaluate the expressions (with side effects) in one mode, and not evaluate them in the other.
  - For runtime evaluation, in *Check\_and\_abort* mode, allow the predicates to be evaluated zero, one, or more times — at the implementation's discretion — which means that the side effects become unreliable, as well as the results, in case the result changes from one evaluation to another.
  - Allow the compiler to assume that contract predicates are side-effect free and allow it to apply code transformations based on this assumption (e.g., not evaluate the same expression twice) — this would mean that a side effect effectuated during the evaluation of a contract predicate is undefined behavior.

## 1.2. Design criteria

{ovr.cri}

We recognize the following four "entities" affected of contract annotations:

1. Programmers and IDEs: for informational purposes.
2. Static analysis: to report in a systematic way execution paths that lead to contract violations.
3. Runtime checks: to detect situation where a contract annotation is violated in every path actually taken at runtime, and abort the execution of the program (or a part thereof) immediately.
4. Compiled programs: contract predicates are potentially evaluated expressions and they may affect the semantics of the programs.

The approach to misbehaving expressions in contract annotations should consider the effect on these three consumers.

We recognize the following three design criteria:

- minimal but extensible scope,
- safety,
- usability.

### 1.2.1. Keeping the scope minimal but extensible

{ovr.cri.mvp}

We want to remain faithful to the objectives outlined in [\[P2182R0\]](#), and called the MVP: while providing a useful — albeit small — addition, remain open for the widest possible range of possible future directions. In consequence, we try to avoid specifying semantics for aspects that remain controversial. Where two possible future directions exist, we want to remain open for both, as long as the result remains sufficiently useful. For aspects that we do not want to define, we prefer making them ill-formed to "Undefined Behavior" (UB) or "Ill Formed, no diagnostic required" (IF-NDR). This is to spare the programmers the runtime surprises, and to save the syntax space for future extensions.

For the cases where we have to commit to syntax, but where no satisfactory semantics exist, we have a couple of options:

- make the program ill-formed with no diagnostic required (IF-NDR) when some construct exists,
- define the effects as undefined behavior (UB) when some construct is evaluated,
- define the behavior as unspecified or implementation-defined,
- provide arbitrary semantics, even if they are surprising to the programmers.

Defining a construct as ill-formed with no diagnostic leaves the door open for defining it as ill-formed (with a required diagnostic) in the future. Defining the execution of a construct as undefined behavior allows the compiler to take arbitrary action, unanticipated by the Standard: these actions can be dangerous, like eliminating arbitrary portions of code at arbitrary locations, but they can also be helpful in detecting bugs, such as launching a debugger, or emitting runtime diagnostics. Making the behavior unspecified or implementation-defined reduces the scope of possible outcomes: we do not get arbitrary negative effects, but we also do not get arbitrary positive effect that we could not list in the Standard but which would help detecting bugs. Finally, when we assign arbitrary fixed semantics, where no good intuitive semantics exist, we risk rendering programs with semantics unintended by the programmers and prevent the tools from detecting these bugs.

### 1.2.2. Safety

{ovr.cri.saf}

We consider contract annotations to be a feature whose primary goal is to increase safety. However, we observe that there exists interpretations of what that means, also among the WG21 experts, so this statement requires more clarification.

On 2020-10-06 SG21 took the following poll.

Poll: we agree to progress towards contract checking to enforce software safety first, and enable assumptions of injected facts at a later time.

|    |   |   |   |    |
|----|---|---|---|----|
| SF | F | N | A | SA |
| 7  | 4 | 2 | 0 | 1  |

This was in the context of discussing the property of "assuming" the truth of contract predicates for the purpose of generating a more efficient code at the expense of modifying the program semantics. This was the most controversial aspect of [P0542R5]. This property, however, had no particular relation to side effects in contract predicates, so it cannot guide our design.

We adapt the definition of *safety* provided in [P2026R0]:

Safety is an emergent property of systems, measured statistically, and regulated by policy. The prerogative to decide if something is "safe" is usually reserved by governmental organizations.

In particular, it recognizes that *safety* is a property of systems rather than programming languages or individual language constructs. What we can aim for in the language design is how the new feature — in combination with other features and development tools and procedures — contributes to the system safety.

In particular, we do not subscribe to the view that reducing the number of places in the Standard that declare some situation as undefined behavior — potentially adding arbitrary behavior unintended by the programmer — automatically increases the safety of the entire systems. Instead, we try to minimize the situations where the program semantics can have effects different than these inferred by humans from inspecting the code.

Given that we aim for safety in the context of the language that generally prioritizes performance and portability over safety, for this feature we expect a different trade-off between safety and efficiency/expressibility than for other C++ features; therefore we are prepared to compromise the uniformity with other features, and not necessarily give in to arguments like, "treat contract predicates as any other expressions."

### 1.2.3. The usability criteria

**{ovr.cri.usa}**

Naturally, we expect the contract support facility to be usable. As a practical criterion, we offer a question: can the Standard Library functions be used as contract predicates? We recommend three specific predicate examples.

```
void test1(std::vector<int> vec)
    PRE(!vec.empty());
```

This is somewhat artificial because it takes a vector by value. This is to avoid (for now) the question of indirection. The questions asked by this example are:

1. Do Standard Library functions require a new annotation (a new specifier, a new attribute) to be usable in contract annotations?
2. Even if declarations remain unaffected, do we impose a library-wide requirement on the Standard Library implementations that certain functions must meet additional criteria (such as having no side effects)?
3. Is the Standard Library implementation allowed to do other things on the side than just producing the value? In particular, in debugging mode, is the implementation allowed to do logging or signalling by other means (such as a pop-up window) that the library is used against its contract?

```
void test2(std::vector<int> const& vec)
    PRE(!vec.empty());
```

This example additionally asks if indirection through a reference is allowed. This question is not really tied to the Standard Library.

```
void test3(auto begin, auto end)
    PRE(std::all_of(begin, end, is_cute{}));
```

This is to additionally check the following:

1. Can the standard algorithms be used in predicates?
2. Can any form of callback (even a user provided class or a function pointer) be used in predicates?
3. Can indirection through iterators be used? (we are inspecting the collection that we do not own.)
4. Can we use the Standard Library iterators, given that some implementations offer an "iterator debugging" mode, where additional state is maintained in the iterators, things are checked and errors reported via side effects such as displaying text.

## 2. Discussion

**{dis}**

### 2.1. Classification of side effects

**{dis.cls}**

Our intention is that programmers would never put contract predicates whose side effects affect the logic of the program, such as:

```
void g()
    PRE(start_device_if_not_started()) // discouraged!
{
    use_device();
}
```

Nor contract predicates whose side effects affect the program correctness -- understood as compliance with contract annotations:

```
void f()
    PRE(less_than_100_calls_so_far()); // discouraged!
```

However, [P2388R4](#) lists side effects that could be considered benign:

1. Logging, which never affects subsequent computations.
2. Modifying private mutable data members for the purpose of caching function results.
3. Using mathematical functions from `<cmath>`, which store error results in global (thread-local) variable `errno`.
4. Performing scoped locking inside the function, which may affect the execution of other threads.
5. Triggering a contract violation handler when runtime-checking the precondition of the function called in the predicate.

These benign side effects do not affect the mental model for contract annotations: from the programmer's perspective the predicates are still logical predicates not interfering with the program logic. More, it is often impossible for programmers to even know if the function they use are referentially transparent. For instance, the specification of `std::vector<T>::size()` does not prevent the implementations from performing side effects, such as logging. And in fact implementations of STL do insert things like safety checks and conditional logging to operations that are conceptually referentially transparent, in debugging mode. For instance, consider `_ITERATOR_DEBUG_LEVEL` in Visual Studio.

Finally, there are side effects that are only observable inside the invoked function, and cannot possibly have any effect on the calling program:

```
bool is_greater_than_one(int i)
{
    int num = 0;
    ++num;
    return i > num;
}
```

This is what [P2680R0](#) would call "side-effect-free when seen from the outside of the cone of evaluation".

The C++ Standard currently has the following definition of "side effect" in [\[intro.execution\]](#):

Reading an object designated by a `volatile` glvalue ([\[basic.lval\]](#)), modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a volatile glvalue is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the `volatile` access may not have completed yet.

Based on the above, we could introduce the classification of side effects:

1. Any side effect, as defined by the C++ Standard.
2. Any side effect *visible outside* the predicate. For instance, creating a local automatic variable and mutating it inside the predicate does not count.
3. A side effect that does not affect the program logic.

The first one is formally defined. We could also provide a definition for the second.

The third class depends on the notion of "program logic" which only the programmer is aware of. For instance, logging into a file may or may not affect the program logic, depending on whether the program later decides to read the contents of the log.

## 2.2. Why side effects are problematic

**{dis.why}**

Why are side effects an issue? If there are no side effects in contract predicates (and predicates don't evaluate to `false`) it is indistinguishable whether a predicate is evaluated or not, or how many times it is evaluated. Even if translation modes are changed, the only difference is performance and the occasional program abort. But in the presence of side effects, switching from *No\_eval* to *Eval\_and\_abort* changes the behavior of the program. In the very extreme (and unlikely) case we can have two binaries doing arbitrarily different things compiled from the same sources but with different translation mode. This problem is not new: the users of C-style `asserts` have been dealing with it for decades. But the context is somewhat new:

1. We are now designing a language feature with new opportunities for additional semantics.
2. C-style `asserts` have never been used in function interfaces, which poses a new implementation challenges: how to perform indirect function calls, where the check belongs neither in the caller nor in the callee.

Given that the primary goal of the contract predicates is to represent a predicate in the logical sense, it is difficult to reason about them when they are not predicates: difficult both for humans and for tools such as code analyzers. Regarding runtime evaluation, if you consider code like this:

```
bool pred(int i); // no reachable definition
void foo(int i) PRE(pred(i));

void bar(int i)
{
    if (pred(i))
```

```
    foo(i);
}
```

Given that we checked that predicate `pred()` holds, we might want the compiler to skip the second check, as we already convinced ourselves and everyone else that the precondition holds. The program is already guaranteed not to execute the body of `foo()` when `pred(i)` is false.

Next, in *Eval\_and\_abort* translation mode, a possible implementation of function `foo()` above is:

```
void foo(int i)
{
    if (!pred(i)) {
        std::cerr << " contract violation in " << __FILE__ << " at " << __LINE__ << std::endl;
        std::abort();
    }

    /* the function body */
}
```

This implementation will work correctly even if `foo()` is called indirectly via a pointer. But when called directly, it may still be desirable for the compiler to insert a similar check in the caller:

```
void caller(int i)
{
    if (!pred(i)) {
        std::cerr << " contract violation in " << __FILE__ << " at " << __LINE__ << std::endl;
        std::abort();
    }

    foo(i);
}
```

The goal would be to have the file name and the line of the caller to be printed, because the bug is in the caller rather than in the callee. But if `pred()` has side effects, this results in evaluating them twice. It should be noted that an alternative implementation using a thunk does not have this problem.

Finally, one special case of a side effect should be given special consideration: when a contract predicate modifies the value of a function argument. This is where checking the condition can actually alter the condition result:

```
int fun(int i)
    PRE(++i <= 0);
```

[\[P0542R5\]](#) explicitly stated that this situation was undefined behavior. This problem is not limited to preconditions: a postcondition modifying the return value would pose similar difficulties.

### 2.3. Treating non-predicate predicates as errors

**{dis.err}**

In order to guarantee that contract predicates are referentially transparent, we can simply make the program ill-formed when a predicate cannot be demonstrated to be referentially transparent. Conceptually this looks simple, but allows us to use very few predicates. Arithmetic operations on built-in types are fine:

```
void f(int i, int* p)
    PRE(i > 0)           // demonstrably referentially transparent
    PRE(p != nullptr); // demonstrably referentially transparent
```

But the following is not:

```
void g(int i)
    PRE(some_array[i] > 0); // accessing object that can be changed externally
```

The above predicate is not referentially transparent: the result does not depend on function arguments. As a consequence, if you verify that the function's precondition holds for value `i`, you cannot be sure that it also holds for the same value the second later:

```
g(0); // checked that the precondition is satisfied
x();  // modifies `some_array`?
g(0); // precondition may not be satisfied
```

The same problem applies to dereferencing a pointer, or making an indirection through a reference, because a pointer is no different than an index to a global array. And since we often take arguments by a reference (to `CONST`), all of them are not suitable.

Interestingly, the following predicate could be considered valid:

```

struct X { int j; };

void h(int i)
    PRE(--(int&)X{i}.j != 0); // ok: modifies internal object

```

While it mutates the state of an object, this object is internal to the predicate: its life started after the evaluation of the predicate, and ended before the end of the evaluation of the predicate. No-one will observe this modification. For the same reason, the following predicate, however useless, would be valid:

```

void h(int i)
    PRE((delete new int, true)); // ok: no side effects are leaked

```

One relaxation to the above rules that would make the solution more usable would be to only require that predicates have no external side effects, but allow them to read the entire state of the program (not only constants or function parameters).

But this is not much. This doesn't allow us to write a simple precondition like:

```

void read(std::vector<int> const& vec)
    PRE(!vec.empty()); // error: calls a function, `vec` is a reference

```

It is not clear how to approach the problem of indirection.

Can functions be called in this model? There is a number of ways to allow this, each with its set of problems.

One solution would be to only allow functions that are themselves declared as, and compiler-checked to be, referentially transparent. But this would leave very little functions usable as predicates, because there are no functions declared as referentially transparent today. And it may not be possible to get many in the future: even functions as basic as `std::vector<T>::size()` may need to do logging for debugging purposes. Even if adapted, it has another cost: we put another fracture in the family of expressions, and require even more specifiers to be put on function declarations. This approach is similar to the one adapted for Transactional Memory support in [\[N4302\]](#).

We could do what [\[P2680R0\]](#) suggests and allow any function whose definition is reachable and proven by the compiler to be referentially transparent.

This still has its problems. First, again, even `std::vector<T>::size()` might fail the test. Second, should the following be considered referentially transparent?

```

bool is_nonzero(int i)
{
    if (false)
        std::cout << "side effect"; // never invoked, but present in code

    return i != 0;
}

```

But most importantly, it is incompatible with the practice characteristic of C++ to put function declarations in header files and function bodies in "cpp" files. Second, it now ties the correctness of the program with things that are traditionally considered implementation details in C++: where the definition of the function is located and if it performs non-referentially-transparent operations.

Imagine that a library author, call her Lena, offers the following function in the library interface header file:

```

inline int lib3::mul_add(int a, int b, int c)
{
    return a * b + c;
}

```

Lena has no intention for this function to be used in contract predicates. It is just meant to be an ordinary function. Now, one of many library users, call him Paul, decides to use the function in his contract predicate:

```

void g(int a, int b, int c)
    PRE(lib3::mul_add(a, b, c) < 1000)
{ /* ... */ }

```

This works (compiles) for Paul, because `mul_add()` happens to meet all the necessary criteria today, and the compiler can see the function body.

Now, in the new revision of the library Lena decides to move the definition of the function into the compiled cpp file, and only leaves the declaration in the header. All Lena's tests pass. But Lena doesn't know about Paul, and that he is using the function in a contract predicate. She never even gave a promise that her function would work in contract predicates, this was never the intent, and it still is not. There is no reason to suspect that this innocent change should break anyone's code.

Paul starts using the new library, and his functions fail to compile. Paul has no control over the declarations of `mul_add()`. He cannot change it. Worse still, function `g` may come from another third party vendor, and Paul may not have the control over `g` as well. The program is now broken, with no easy fix.

The library authors cannot know how their functions are used in user code, so they would have to assume the worst (usage in contract predicates) and couldn't safely change the location of function definitions. A similar reasoning applies to side effects. Library authors would not be able to safely add any side effects to their functions (other than the modification of local variables) because this could break their users' code. Adding a logging statement to my function might break a build.

It should be noted that even with any of these these extensions in place some intuitive uses of contract annotations will not work. For instance, when functions are called through function wrappers:

```
int algo(function<int(int)> op, int i)
    PRE(op(i) > 0); // error: function::operator() is not declared as referentially transparent
```

A function wrapper like `std::function` must be prepared to handle both kinds of functions, assigned at runtime. Therefore, the wrapper itself cannot be declared as referentially transparent.

### 2.3.1. Possible extension: allow side effects

{dis.err.rlx}

Finally, it should be noted that if this strict approach is applied, it remains a possible future direction to simply lift it in the future and allow side effects in contract predicates. This door remains open if we take the conservative approach for the MVP.

## 2.4. Unchecked declarations of referential transparency

{dis.uns}

Another relaxation to the above model is to allow functions that are declared `[[unsequenced]]` as described in [\[N2956\]](#) (which has been approved by WG14 for C22). The `[[unsequenced]]` means that the author allows the compiler to treat the function as referentially transparent — have the freedom to replace the call to the function with its result — and the compiler isn't required to verify this: it is undefined behavior if the function body doesn't satisfy the requirements of referential transparency.

What is gained with this is that functions like `std::vector<T>::size()` can be declared as `[[unsequenced]]` and usable in contract predicates, even if they perform logging: the author allows these side effects to be erased.

## 2.5. Treating side effects as undefined behavior

{dis.ube}

Another approach to the problem of side effects in contract predicates is to impose no syntactic constraints on the predicates, but instead call it undefined behavior when the evaluated contract condition has a side effect. This is equivalent to saying that by putting an expression inside a contract annotation, the programmer allows the compiler to treat the expression as if it were referentially transparent.

This has an unsettling consequence of introducing a yet another UB, in the context of a safety feature such as contract support. On the positive side, this allows the implementations to evaluate the contract predicates arbitrary number of times. More, it encourages the implementations to emit diagnostics for side effects planted in contract predicates. The variant of this path is to make the program ill-formed, NDR. This enables the potential future direction of making side effects ill formed.

Another future direction is to assign a well-defined behavior to side effects: for instance, apply the C-assert model.

While term "undefined behavior" may be concerning, it should be noted that here we are talking about a different situation than the one that caused controversies over [\[P0542R5\]](#) (the "C++20 contracts"). Aggressive code transformations based on undefined behavior caused by contract annotation violations were, and remain, a source of concern. This is because contract annotations give the exact information that the optimizers need, and optimizations like this are of interest to users and are implemented in the compilers. The danger here stems from the interaction between UB and code optimizations. This has been described in [\[P1728R0\]](#). We can call it "contract-based optimizations".

On the other hand, regarding the UB resulting from side effects in contract predicates — we can call it "side effect elision/duplication" — there is no known way for them to be used in optimizations. So, the concern of the same nature does not apply here. The remaining concern is that with appearing and disappearing side effects. And the risk is even smaller when the predicates do not contain side effects. Note that the problem with "contract-based optimizations" occurs when contracts themselves are correct and only the programmers misuse the components (e.g., call functions out of contract). On the other hand, problems with "side effect elision/duplication" arise only if contract annotations themselves are fishy.

## 2.6. Allow side effects

{dis.sem}

Another possibility is to allow just any expression in contract predicates. Then we have to specify the semantics in the two translation modes: *No\_eval* and *Eval\_and\_abort*. Mode *No\_eval* is simple: we guarantee that the predicates are never evaluated: this implies no side effects are effectuated. This mode guarantees three things: performance, no UB resulting from evaluating the expression, and no false negatives in case when the author puts the wrong expression in the contract predicate.

For the other mode — *Eval\_and\_abort* — we can see two options. It can express one of the two guarantees:

1. We guarantee that the program will not continue upon value `false`, but we do not guarantee that the side effects of the expression will be observable, or if the expression is called twice. We will call it *abort guarantee*.
2. We guarantee the same as C `assert()` does: that the expression is simply evaluated, single time, and its side effects are guaranteed as for any other expression in the language. We will call it *eval guarantee*.

So, our primary question is what the user expects from *Eval\_and\_abort* mode: program abort upon a detected bug, or conditional evaluations of expressions? For instance, should the following use case be guaranteed to work?



```

bool mode_eval_and_abort = false;

void log_mode()
  PRE(mode_eval_and_abort = true)
{
  clog << "running program in mode " << (mode_eval_and_abort ? "Eval_and_abort"sv : "No_eval"sv);
}

```

The argument in favour of using *eval guarantee* (side effects executed, exactly once) is simplicity. If there is any bug in the application — not detected by contract annotations, or caused by contract annotations — compiled in *Eval\_and\_abort* mode, when you need to debug the problem, it is easier when you can rely on the fact that the side effects from contract predicates are evaluated as any other expression. This is a more intuitive model, in the spirit of an imperative language, such as C++. It follows the existing practice with `assert()`: it allows side effects, but a lot of advice comes with it, saying that side effects in the predicate cannot be relied upon.

The arguments in favor of duplicating or eliminating side effects are:

1. Leaving room for implementations that might need to execute predicates twice in some cases.
2. Exerting a pressure on programmers, so that they provide external-side-effect-free contract predicates.
3. Avoid redundant runtime checks.

Some implementation strategies may need to evaluate the same predicate in a precondition twice. For direct function calls, an implementation can easily insert the instrumentation code in the caller. This is desired as it gives better diagnostics (the file name and line number of the caller who violated the precondition). However, this is impossible when a function is called indirectly, either through a pointer or `std::function`: from the pointer signature we do not know if a function called has a precondition or not. To address that case, one thing an implementation could do is to compile the pre- and post-condition checks into the function body. This would give the result that the pre-/post-conditions are checked normally when the function is called through an indirection, but are checked twice when the function is called directly: once in the caller, and once inside the function body. We may want to enable such implementation strategies. The consequence for the programmer is that when the predicate has side effects, these effects occur twice.

To be more specific, when we say that side effects are duplicated, we mean calling the full sequence of preconditions (or postconditions) twice, rather than calling the individual contract predicates twice. That is, the execution of the following function with side-effect-full preconditions

```

bool pred(int i, int c)
{
  std::cout << c << " ";
  return true;
}

int fun(int i)
  [[pre: pred(i, 1)]]
  [[pre: pred(i, 2)]];

```

can render "1 2 1 2 " rather than "1 1 2 2 ".

The implementation might want to provide a "mixed mode" where different translation units are compiled in different modes. For the MVP, we define this situation as IF-NDR, but in the future we may want to allow it. At this point we do not know the implementation difficulties that might arise from the mixed modes, the necessity to duplicate side effects in certain cases might be one of the results.

The mere possibility that the side effects of contract predicates can disappear in *Eval\_and\_abort* mode — provided that the programmer is aware of this — should be a strong incentive not put any "essential" side effects (e.g., logging is not such essential side effect) in their predicates. As we cannot easily prevent side effects statically in C++, this might be our best bet. The *abort guarantee* also does not rise a false expectation that preconditions and postconditions are always evaluated or skipped in pairs:

```

int counter = 0;

void f() noexcept
  PRE(++counter == 1)
  POST(--counter == 0);

```

This strange (and not recommended) combination of pre- and post-conditions holds when both pre- and post-conditions are skipped, as well as both pre- and post-conditions are evaluated. However, in the future, we might want to add a translation mode where only preconditions are evaluated and postconditions are skipped. This is a very useful mode representing a practical trade-off between safety and efficiency. Preconditions are violated far more often than postconditions. This is because the function caller is not the author of the contract, so they may not know the precondition, whereas the postcondition needs to be satisfied by the author of the function who knows the contract. This mode has been implemented in ADA and Eiffel:

- ADA has pragma `Assertion_Policy(Pre => Check)` (see [GNAT](#)).
- Eiffel has level `require` (see [Eiffel](#)).

This translation mode might break user code, if there was a guarantee that both pre- and post- conditions are either both evaluated or both skipped in any mode.



Given that the primary goal of contract predicates is to provide the *abort guarantee*, the provision to evaluate the predicate zero times could allow the implementations to skip the redundant evaluation of the predicate if its result is already known:

```
bool pred(int i); // no reachable definition, may contain side effects

void foo(int i)
    PRE(pred(i));

void bar(int i)
{
    if (pred(i))
        foo(i);
}
```

Given that we have evaluated `pred()` just before the function call, in the `if`-statement, we already know the result, so calling it again before the function body is redundant.

C++ has other contexts where the side effects that would have otherwise been expected are removed:

1. Short-circuiting in logical operations (guaranteed removal).
2. Copy elision (optional removal).

## 2.6.2 Future extensions

{dis.sem.fut}

Also, it should be noted that the side effect elimination/duplication, if deemed useful, need to be added in the MVP and cannot be added later, because then it would be a breaking change for people who rely on these side effects. Conversely however, if we allow duplication/elimination of side effects in contract predicates to the MVP, we can tighten the rules later, at the cost of compromising contract violation messages, or compromising performance.

## 2.7. Static analysis in the presence of side effects

{dis.mix}

One could ask if static analysis based on contract annotations must require that contract predicates be referentially transparent. After all, if we can force the compiler to determine if a predicate is referentially transparent, the static analyzer can do that as well. The static analyzer, upon encountering a predicate, could first determine if the predicate has external side effects. If so, the predicate is ignored, otherwise it is consumed by the analyzer.

Such solution, however, loses one important guarantee. If referential transparency is enforced in a contract annotation, the presence of the contract annotation in the code, is a visible guarantee given to programmers who review the code that the predicate is consumable by the analyzer. If this responsibility is assigned to the compiler, the analyzer, which usually reaches resource limits, can be relieved of this part.

The compile-time enforcement of referential transparency, if doable at all, also helps prevent bugs where the programmer inadvertently puts a side effect into their function, for instance by naming the wrong function. Such verification is especially required for contract predicates that are meant to be a tool for enforcing both correctness and safety. Additional measures should be then taken to make sure that the tool itself does not compromise correctness or safety.

While it is true that some static analyzers could consume contract predicates even with side effects, or at least some such predicates, enforcing referential transparency, makes the bar lower for static analyzers, and therefore more of such tools would be able to consume the predicates.

A solution that would both allow the above guarantee and the presence of external side effects would be to allow the programmer to flag a contract annotation with the information if referential transparency is guaranteed. In the following example `PRE` indicates that a referential transparency check is enforced, whereas `PRE_RELAXED` indicates no referential transparency enforcement:

```
bool is_nonzero(BigInt i) { i != 0; } // demonstrably free of external side effects
bool is_special(BigInt i);           // potentially has external side effects

BigInt fun(BigInt num, BigInt den)
    PRE(is_nonzero(den))           // ok, static analyzer will consume
    PRE(is_special(den))           // error: potential external side effects
    PRE_RELAXED(is_nonzero(num))   // ok, but static analyzer will not consume even though it could
    PRE_RELAXED(is_special(num))   // ok, static analyzer will not consume
{
    // static analyzer may guarantee that `is_nonzero(den)` is true
    // no other precondition guaranteed by static analyzer
    return num / den;
}
```

With this additional flagging of contract annotations, when it comes to the runtime evaluation of the predicates, we have two options:

1. When the build chain is not using a static analyzer with the properties described above, both groups of predicates — declared with `PRE` and with `PRE_RELAXED` — are runtime-evaluated.

2. When the build chain is using the static analyzer, some predicates declared with `PRE` may be proven to be satisfied by the static analyzer. These do not need to be runtime checked as already proven never to be `false`. The remaining predicates (these marked with `PRE_RELAXED` and the unproven ones marked with `PRE` are runtime-checked.

Such skipping of runtime checks for analyzer-proven contract predicates is in a way similar to `[[assume(x)]]` from [P1774R8] with one essential difference: it is not the user who injects the (possibly incorrect) information; instead the information is inferred by the static analyzer, and safe from human error.

It should be noted that the feature represented by identifier `PRE_RELAXED` can be added to the language independent of the referential transparency enforcement.

### 3. Acknowledgements

{ack}

This paper is a summary of discussion between SG21 members about side effects in contract predicates. Gabriel Dos Reis suggested and explained the approach of making side effects ill-formed. Ville Voutilainen suggested the approach to mix two kinds of predicates described in {dis.mix}. Herb Sutter suggested the test to what extent the Standard Library components can be used in contract predicates.

### 4. References

{ref}

- [Eiffel] — Eiffel Documentation ([https://www.eiffel.org/doc/eiffel/ET-Design\\_by\\_Contract\\_%28tm%29%2C\\_Assertions\\_and\\_Exceptions](https://www.eiffel.org/doc/eiffel/ET-Design_by_Contract_%28tm%29%2C_Assertions_and_Exceptions)).
- [GNAT] — GNAT Reference Manual ([https://gcc.gnu.org/onlinedocs/gcc-4.9.4/gnat\\_rm/Pragma-Assertion\\_005fPolicy.html](https://gcc.gnu.org/onlinedocs/gcc-4.9.4/gnat_rm/Pragma-Assertion_005fPolicy.html)).
- [N2956] — Étienne Alepins, Jens Gustedt, "Unsequenced functions" (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2956.htm>).
- [N4302] — "Technical Specification for C++ Extensions for Transactional Memory" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4302.pdf>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P1728R0] — Andrzej Krzemiński, "Preconditions, axiom-level contracts and assumptions – an in depth study" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1728r0.html>).
- [P1774R8] — Timur Doumler, "Portable assumptions" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1774r8.pdf>).
- [P2026R0] — Ryan McDougall et al., "A Constituent Study Group for Safety-Critical Applications" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2026r0.pdf>).
- [P2182R0] — Andrzej Krzemiński, Ryan McDougall, Joshua Berne, "Contract Support: Defining the Minimum Viable Feature Set" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2182r0.html>).
- [P2388R4] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either *No\_eval* or *Eval\_and\_abort*" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html>).
- [P2521R2] — Gašper Ažman, Joshua Berne, Bronek Kozicki, Andrzej Krzemiński, Ryan McDougall, Caleb Sunstrum, "Contract support — Working Paper" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2521r2.html>).
- [P2680R0] — Gabriel Dos Reis, "Contracts for C++: Prioritizing Safety" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2680r0.pdf>).